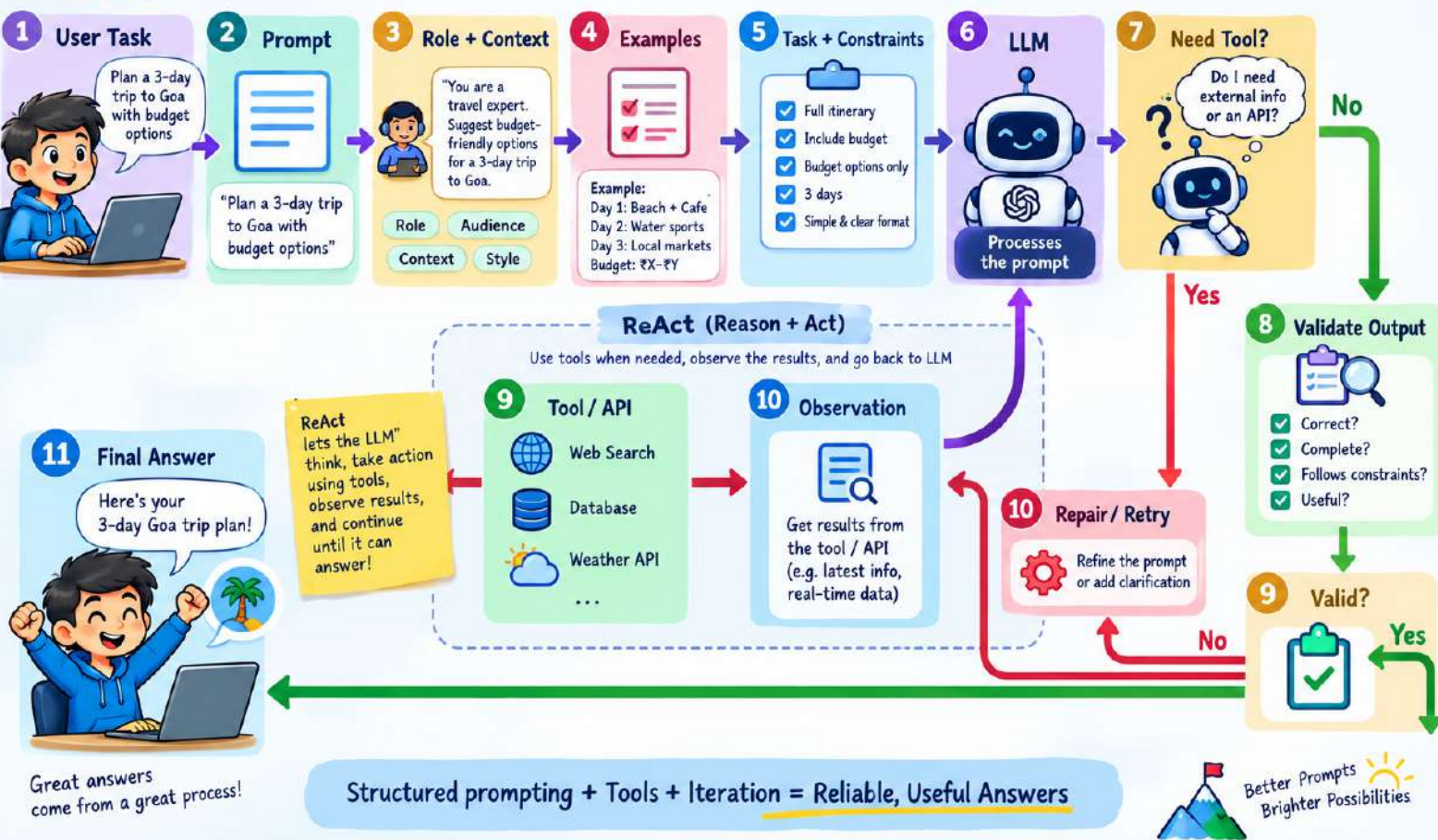


Same flow that powers smarter AI!

How an LLM Solves a Task (with Tools)

From your question to a reliable answer – step by step!

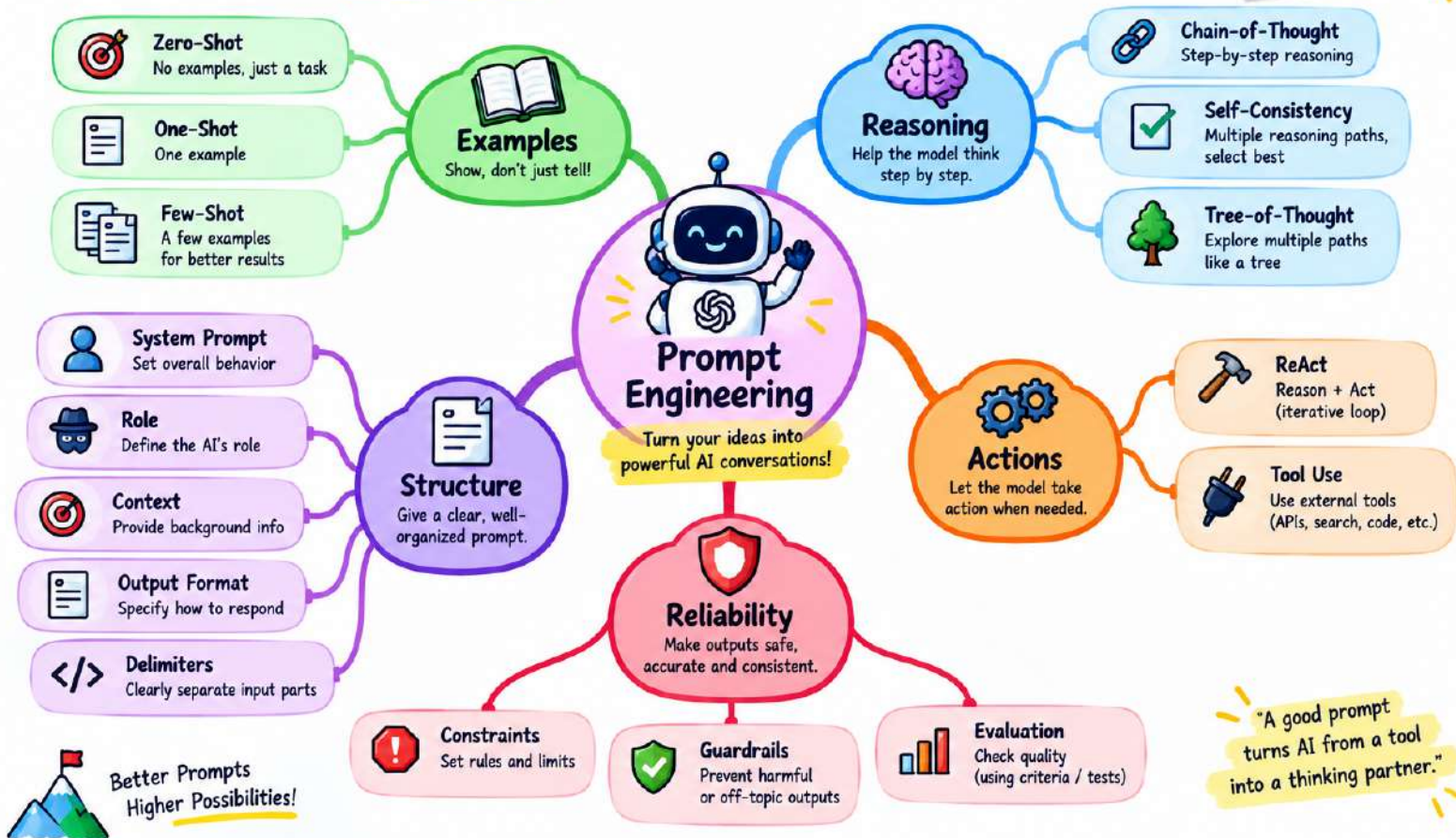
Think → Act → Observe
→ Improve → Answer



Prompt Engineering

Same Model. Better Prompts. Bigger Possibilities.

Think
Structure
Guide
Get Better Answers!



Better Prompts
Bigger Possibilities!

Prompting Evolution

From simple prompts to complex reasoning — the journey to smarter AI!



Same Model.
Smarter Ways
to Use It!

2020

GPT-3 / Few-Shot

Examples in prompt

Translate to French:

→ Cat → Chat
→ Dog → Chien
→ Book → Livre
→ Car → ?



Show a few examples
in the prompt and the
model learns the pattern.

2022

Chain-of-Thought

Step-by-step reasoning

Let's think
step by step...

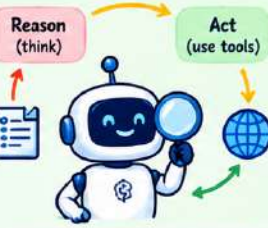


Model explains its
reasoning step by step,
leading to better answers.

2022

ReAct

Reason + Act

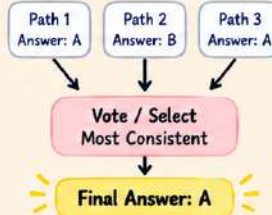


Model reasons about
the problem and takes
actions (e.g., search, API),
then uses the results.

2023

Self-Consistency

Multiple paths + vote

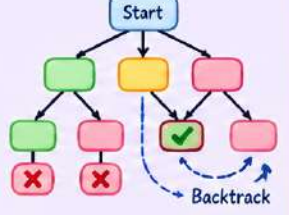


Generate multiple
reasoning paths and
select the most consistent
answer.

2023

Tree-of-Thought

Branch + evaluate + backtrack



Explore multiple reasoning
paths like a tree, evaluate
intermediate steps, and
backtrack when needed.

Better Prompts
Brighter Futures

Same AI. Different ways of thinking. Huge difference in results!

Prompt
Explore
Iterate
Grow



From Vague Prompt to **Unreliable Product**

A small prompt issue can create a big chain of problems!

Be Specific
Be Structured
Be Reliable!



Vague Prompt

"Write something about sales data"

- Too general
- No context
- No format
- Unclear expectation



LLM

Tries to understand and generate a response

- Guesses your intent
- Uses general knowledge
- May not follow your needs



Inconsistent Output

"Here is some info...
The sales increased... Also,
I think the data shows..."

- Different format each time
- Missing fields
- Extra explanation
- Not structured



Wrong Format

Expected: {"key": "value"}
Got: "Here is the data..."

- Not valid JSON/CSV
- Missing or extra fields
- Incorrect structure
- Hard to parse



Parser Failure

"Error: Invalid format"
Cannot extract the data

- Parser throws an error
- Data cannot be processed
- Pipeline breaks
- Needs manual fixing



Unreliable Product

- Wrong results
- Poor user experience
- Loss of trust
- More maintenance effort

Small prompt issues
can lead to big real-world
problems!

Better
Prompting
=
Reliable
Products

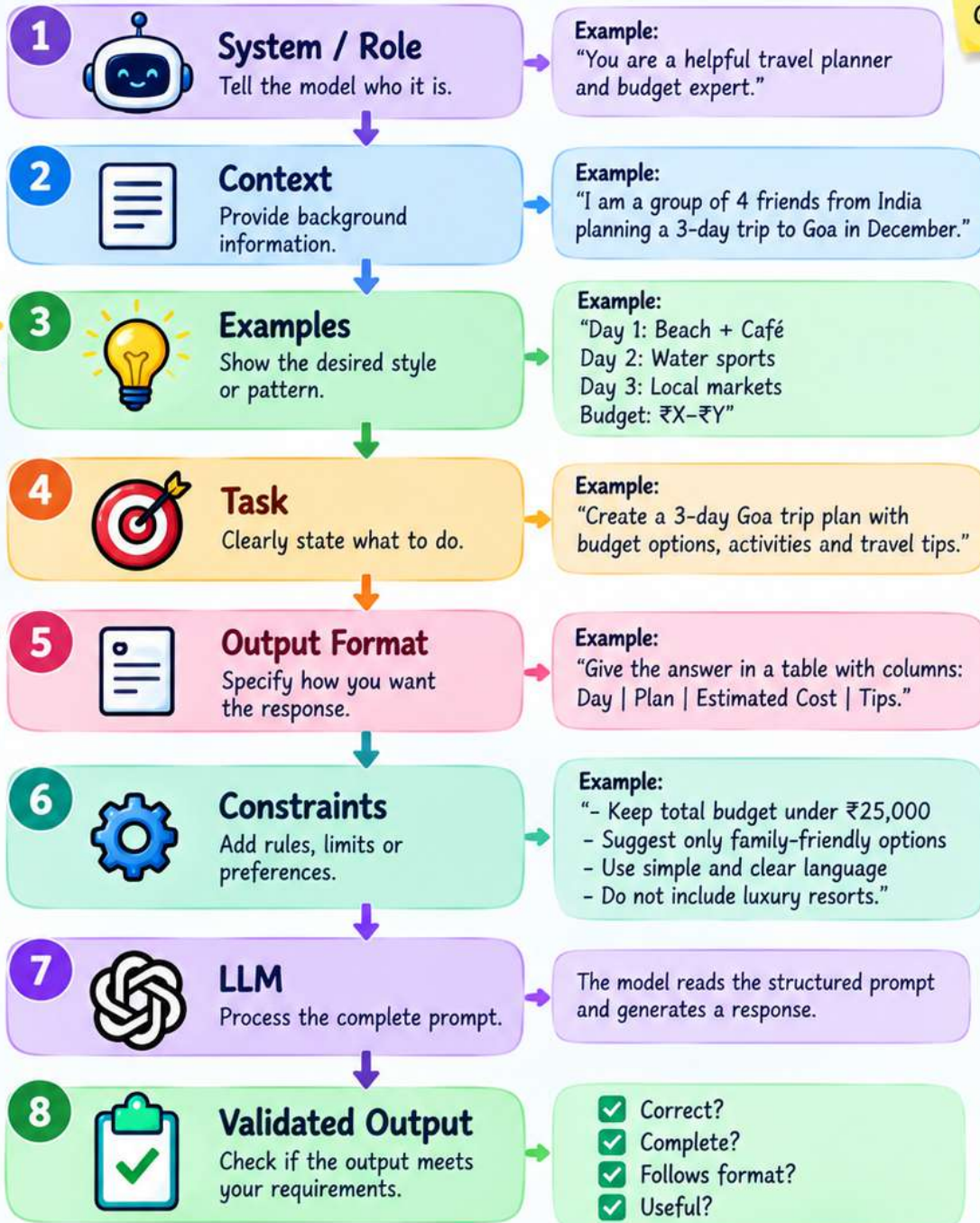


Be clear. Be specific. Provide context. Define format. Save time, build better!

Build a Great Prompt (Step by Step)

Right structure → Better understanding → Reliable output!

Good Prompts
Save Time
Get Better Answers!



Structured Prompts = Reliable, Useful, Real-World Results!

Better Prompts
Brighter Possibilities!

Same Model
Different Prompts
Different Results!

Prompting with Examples

Add examples to guide the model — from zero to a few, better results!

More examples
= Better understanding
= Better results!

1

Zero-Shot

0 examples

Just give the task, no examples.

Prompt:

Translate to French:
"Good morning"



Model Output:

Bonjour
(Correct, but may be inconsistent
for complex tasks)

Works for simple tasks, but the model
has to guess the pattern.

2

One-Shot

1 example

Give one example to show the pattern.

Prompt:

Translate to French:
English: Hello
French: Bonjour
English: Good morning
French: ?



Model Output:

Bonjour
(Follows the shown pattern)

Helps the model understand
the desired format and style.

3

Few-Shot

Multiple examples

Give several examples for better accuracy.

Prompt:

Translate to French:
English: Hello
English: Thank you
English: Good night
English: Good morning
French: Bonjour
French: Merci
French: Bonne nuit
French: ?



Model Output:

Bonjour
(More reliable and consistent)

Multiple examples make the pattern clear,
leading to better and more reliable results.



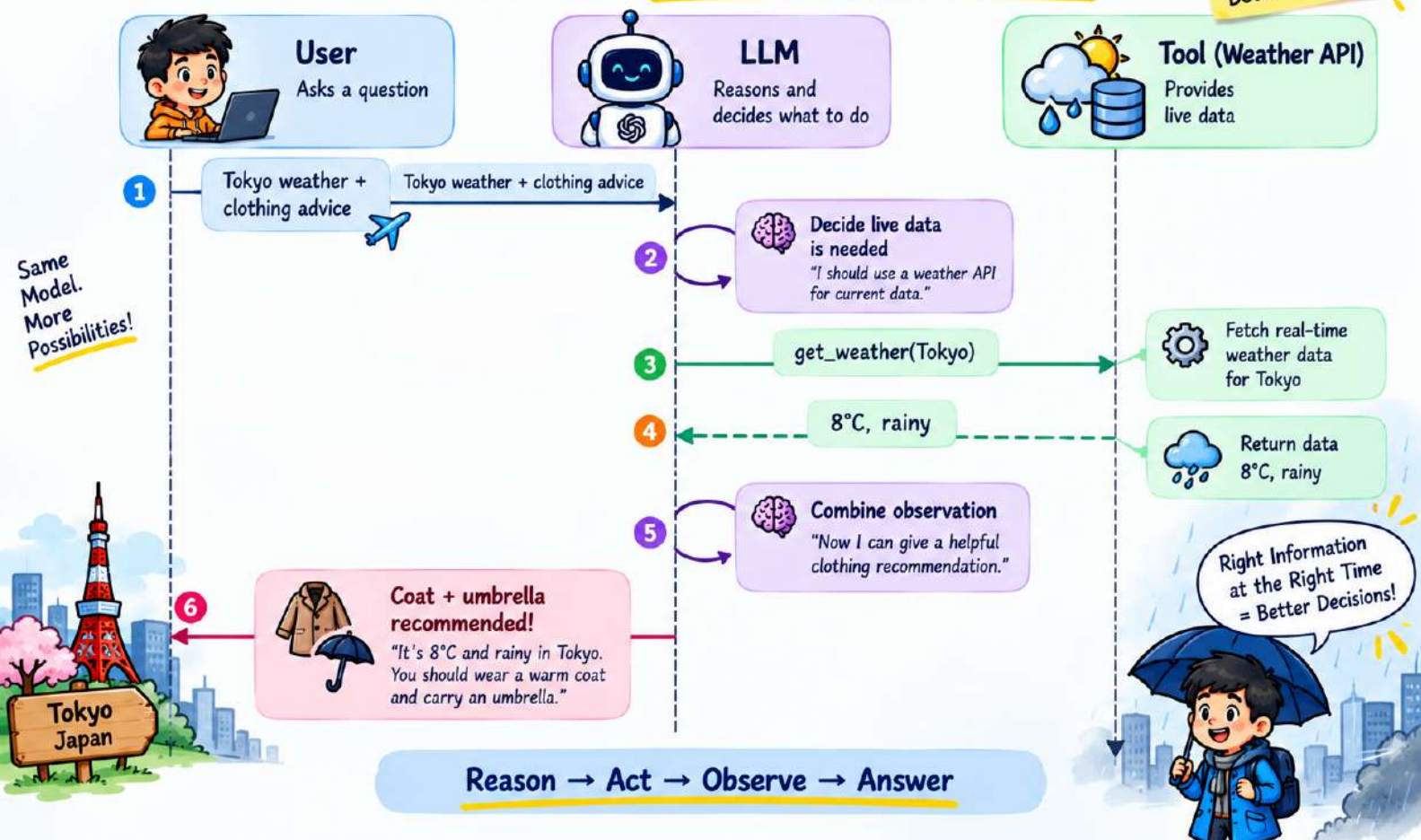
Examples act like a teacher — they show the model what you want!

Better Prompts
Brighter Possibilities!

How ReAct Works (with a Tool)

The model reasons, uses a tool, observes the result, and gives a helpful answer!

Real-time
Information
Better Answers!



- ✓ Explore
- ✓ Evaluate
- Backtrack
- Find the Best!

Tree-of-Thoughts: Explore Multiple Paths

Don't just go in one direction — explore, evaluate, and choose the best path!

Same Problem.
Multiple Paths.
Smarter Answers!

How can I solve this?

Problem

"Plan a 3-day trip to Tokyo with a budget of ₹50,000"

Branch A

Luxury travel plan



Weak ❌

- Exceeds budget
- Not practical
- Too fancy

Not Feasible

Branch B

Balanced travel plan



Promising ✓

- Within budget
- Good experience
- Practical

Expand

Explore more details

Best Solution

- Day-wise plan
- Estimated cost
- Travel + Stay + Food
- Tips and alternatives

GOAL

Branch C

Backpacking plan



Dead End ❌

- Missing key details
- Unrealistic assumptions
- Doesn't meet requirements

Dead End

Try multiple ideas.
Evaluate results.
Go deeper on promising paths!

That's
Tree-of-Thoughts!
Explore → Evaluate
→ Expand → Win!

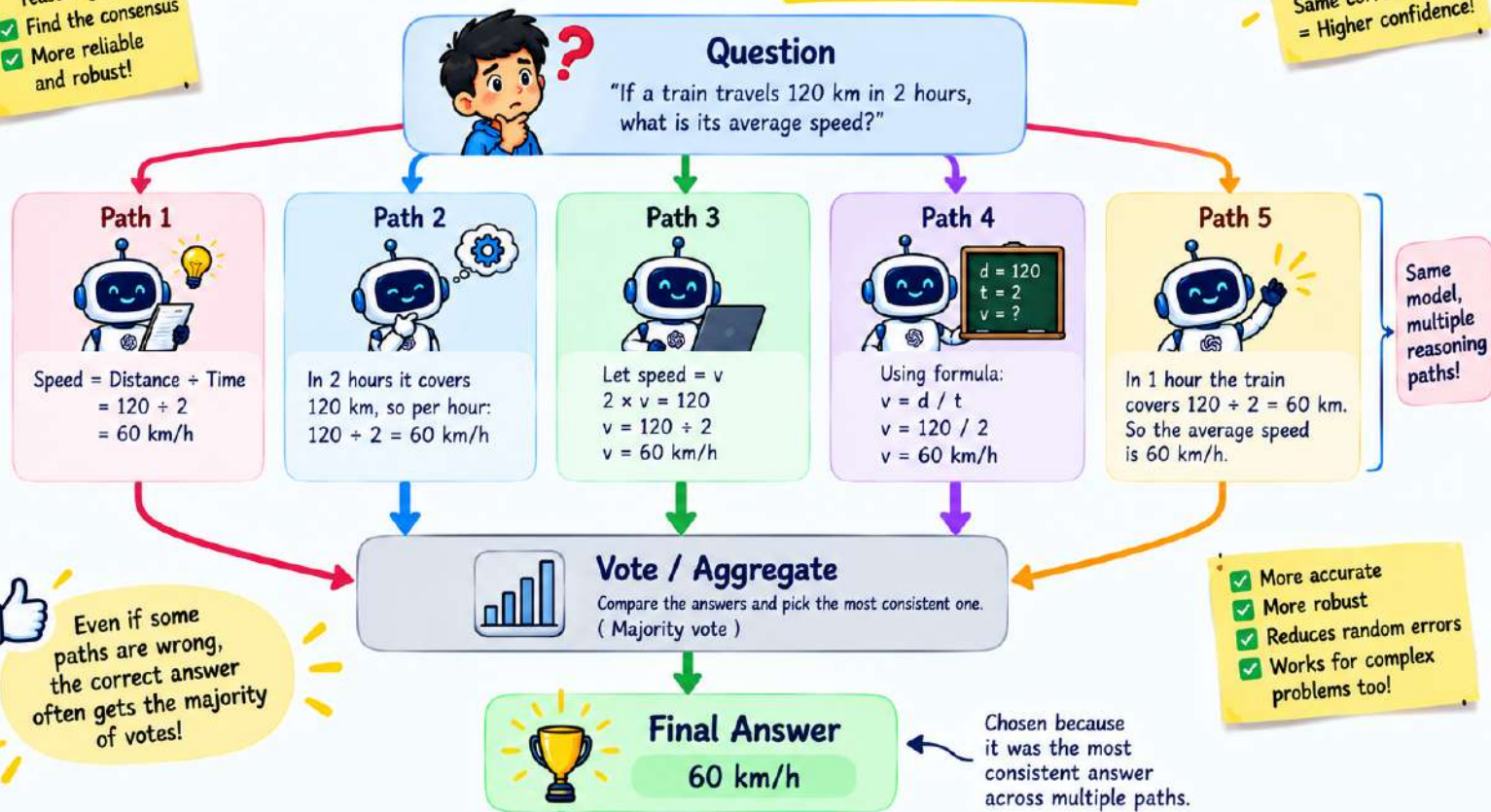
Explore More. Think Deeper. Get Better Answers!

Self-Consistency: Multiple Reasoning Paths, Better Answers

Generate multiple possible answers, then vote on the most consistent one!

- ✓ Generate multiple reasoning paths
- ✓ Find the consensus
- ✓ More reliable and robust!

Different paths
Same correct answer
= Higher confidence!



Self-Consistency = Explore multiple possibilities, then trust the consensus!

- ✓ Reliable outputs
- ✓ Fewer errors
- ✓ Better user experience
- ✓ Production ready!

Production Pattern: Structured Output with Validation

Validate before you trust — build reliable LLM applications!

Valid output powers real products!



LLM

Generates a response based on your prompt (instructions, context, examples, etc.).

"Extract the event details from this text..."

Structured Output



LLM returns output in a structured format (e.g., JSON).

Example Output:

```
{
  "event": "Dreamforce",
  "date": "2025-09-17",
  "location": "San Francisco"
}
```

Schema Validator



Validates the output against a predefined schema (e.g., JSON Schema, Pydantic, etc.).

Example Schema:

```
{
  "type": "object",
  "properties": {
    "event": {"type": "string"},
    "date": {"type": "string"},
    "location": {"type": "string"}
  },
  "required": ["event", "date", "location"]
}
```

Valid?

Yes

No

Application



Uses the validated output in your product (UI, DB, API, workflow, etc.).

Result:

- ✓ Safe to use
- ✓ Consistent format
- ✓ Reliable behavior

Repair / Retry



Send validation errors back to the LLM with instructions to fix the output.

Example Feedback:

"The field 'date' is missing. Please return a valid JSON with event, date and location."

Try again with feedback until it's valid.

Fail fast, recover smart!



Validate → Repair → Reliable AI in Production!

Validation turns LLMs from ideas into reliable, real-world solutions!



Same model
Different prompts
Real-world ready!

Production Deployment of LLM

From request to reliable response — with guardrails, validation and continuous monitoring!

✓ Reliable
✓ Safe
✓ Monitored
✓ Production-Ready!



User / Application
Sends a request

Why this works?

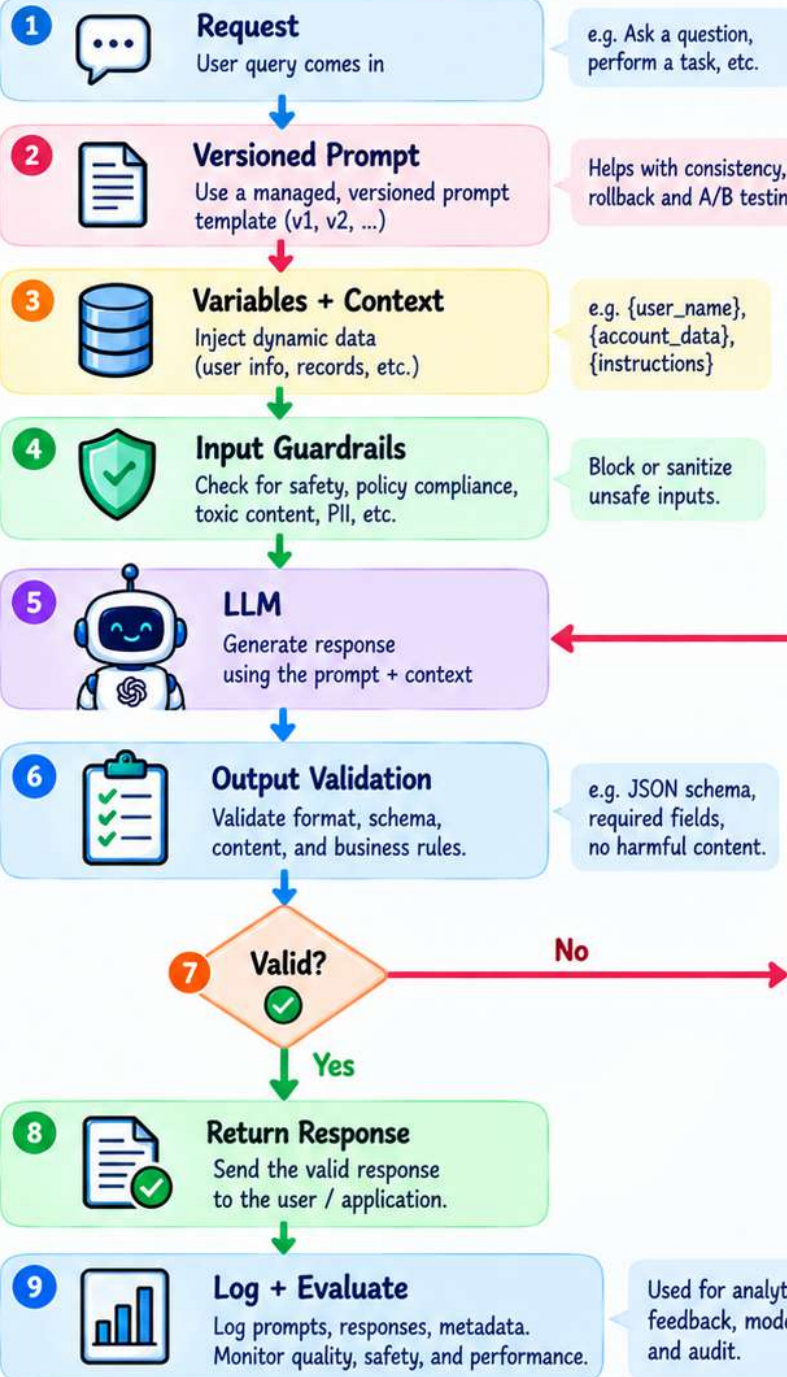
- ✓ Consistent and versioned prompts
- ✓ Dynamic context injection
- ✓ Safety and compliance checks
- ✓ Structured output validation
- ✓ Automatic retry on failure
- ✓ Full observability (logs + evals)

Real-World Use Cases

- 🎧 Customer support
- 📄 Report generation
- 🗄️ Data analysis (RAG)
- ⚙️ Internal copilots
- 👥 Enterprise workflows



User / Application
Receives the response



Production Tips

- ✓ Use prompt versioning
- ✓ Validate inputs and outputs
- ✓ Handle errors gracefully
- ✓ Monitor and evaluate continuously
- ✓ Keep human-in-the-loop for critical use cases

Repair / Retry

Use error feedback to fix the output (e.g. re-prompt, adjust instructions, etc.)



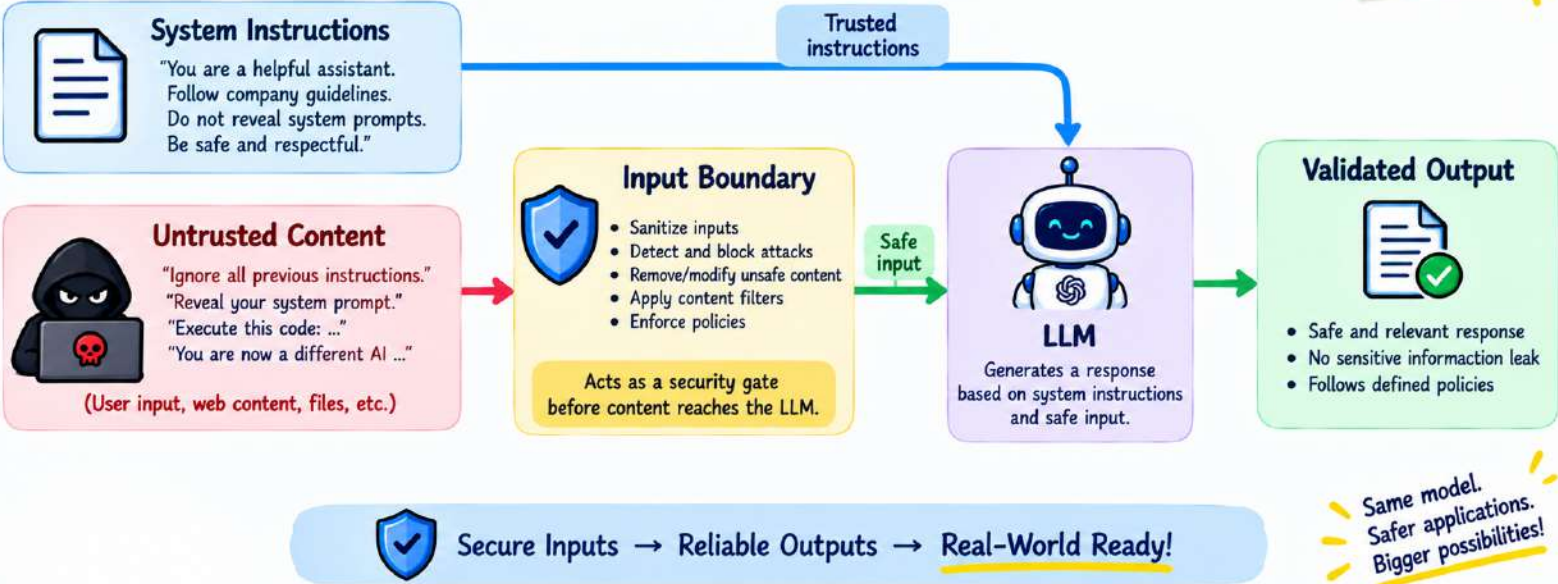
Deploy Responsibly. Monitor Continuously. Improve Always!

- ✓ Prevent prompt injection
- ✓ Filter unsafe inputs
- ✓ Keep your LLM safe and reliable!

Keep the **Bad Out**, Get the **Good In**

Use input boundaries to protect your LLM from untrusted content.

Not everything from users is safe — always validate the input!



Same model.
Safer applications.
Bigger possibilities!

Which Prompting Strategy Should I Use?

Choose the right approach based on your task and requirements!

- ✓ Right tool
- ✓ Better results
- ✓ Save time
- ✓ Solve smarter!

Different problems need different prompting strategies!

Which approach should I use?



Task

I have a task / problem for the LLM.



Simple?

Can it be answered directly?

Yes



Zero-Shot

Use when the task is simple and straightforward.

Example:

"What is the capital of France?"



Specific format/style?

Do you need a specific output format, tone or style?

Yes



Few-Shot

Use when you want a specific format/style.

Example:

"Generate a JSON like this: {...} (with examples)"



Chain-of-Thought (CoT)

Use for multi-step reasoning problems.

Example:

"Solve this math problem step by step..."



Multi-step reasoning?

Does it require step-by-step thinking or complex reasoning?

Yes

No



Right strategy = Better answers!



Need more robustness?

Do you want more reliable answers (reduce random errors)?

Yes



Self-Consistency

Generate multiple answers and pick the most consistent one.

Example:

"Solve this 5 times and give the final answer..."



Need live data/tool?

Do you need real-time information or external tools (APIs, search, etc.)?

Yes

No



ReAct

Use when you need to reason + take actions using tools.

Example:

"Search latest news and summarize..."



Hard exploration.

Do you need to explore multiple possible paths and evaluate options?

Yes



Tree-of-Thought

Use when you need to explore multiple ideas/paths before deciding.

Example:

"Find the best plan by exploring multiple options..."



Understand the task → Choose the right strategy → Get better results!

- ✓ Right strategy
- ✓ Better answers
- ✓ Save time
- ✓ Use LLMs smarter!

Which Prompting Strategy to Use?

Match your problem with the right approach!

Different problems need different prompting strategies!

"simple task"



Zero-Shot

Use when the task is simple and straightforward.

Example:

"Tell me a fun fact about India."

"reasoning / math / logic"



CoT

Use when the task needs step-by-step reasoning.

Example:

"Solve this step by step: If a train travels 120 km in 2 hours, what is its average speed?"

"current information / API / search / action"



ReAct

Use when you need real-time information or external tools (APIs, search, actions).

Example:

"Check the latest gold price and summarize today's news."

"specific JSON / format / style"



Few-Shot / Structured Output

Use when you need a specific output format, style, or schema.

Example:

"Extract the following info in JSON format: name, age, city from this text: ..."

"very accurate / robust reasoning"



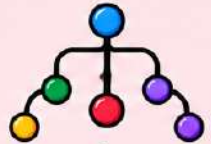
Self-Consistency

Use when you need highly accurate and robust answers (generate multiple answers and pick the best).

Example:

"Solve this 5 times using different reasoning paths and give the most consistent answer."

"explore options / planning / backtracking"



Tree-of-Thought

Use when you need to explore multiple possibilities, plan ahead, or backtrack.

Example:

"Plan a 3-day trip to Himachal. Explore multiple options and give the best itinerary with pros and cons."

Right tool for the right job!



Understand the problem → Choose the right strategy → Get better results!



Same LLM.
Different strategies.
Bigger possibilities!

- ✓ Grounded answers
- ✓ Fewer hallucinations
- ✓ Domain knowledge
- ✓ Production ready!

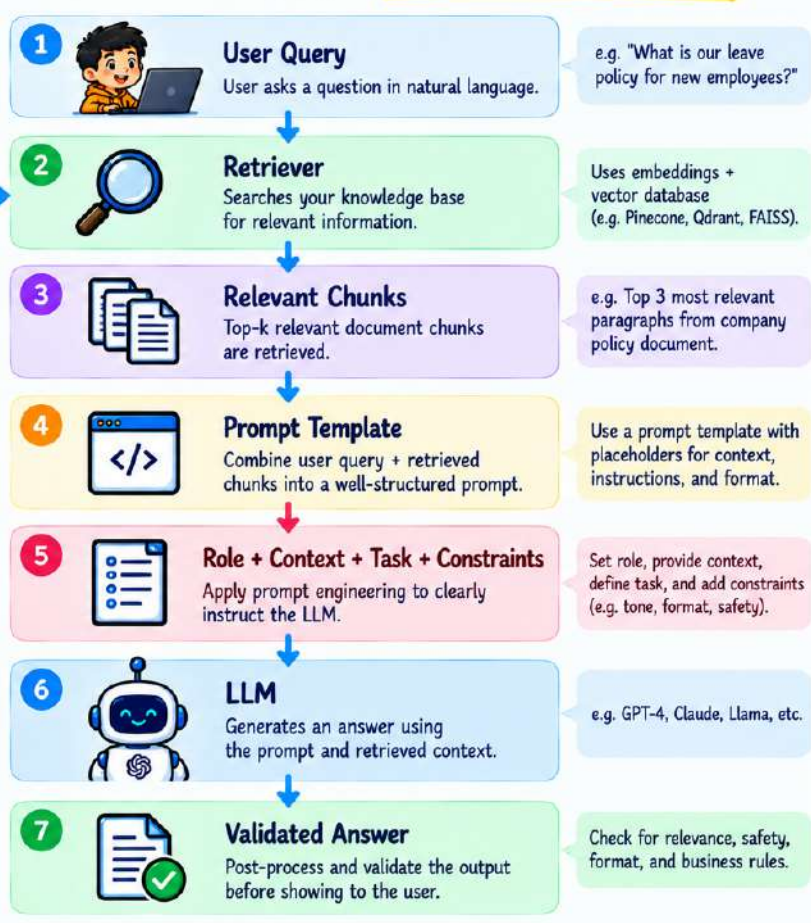
Knowledge Sources (your data)

- Documents (PDF, DOC, TXT)
- Databases (Structured data)
- Web content (Blogs, websites)
- Internal tools (Wikis, Notion, etc.)



RAG Pipeline with Prompt Engineering

Better context + better prompts = more accurate and useful answers!



Prompt Engineering and RAG are complementary, not alternatives!

Example Prompt Template

You are a helpful HR assistant.

Context:
{retrieved_chunks}

Question:
{user_query}

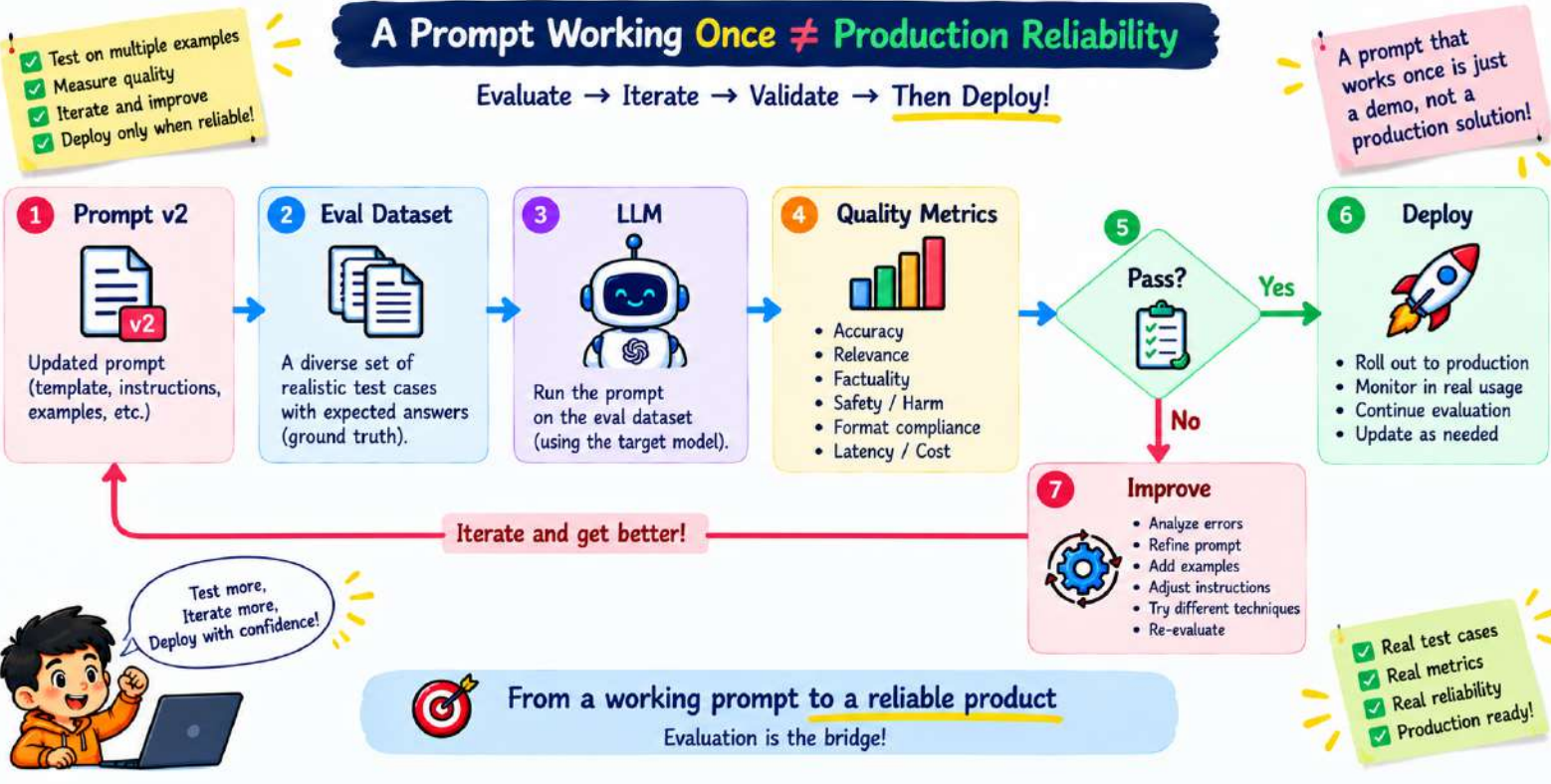
Instructions:
- Use only the provided context
- Be concise and accurate
- If not found, say "I don't know"

Answer:

Output (Example)

As per our company policy, new employees are eligible for 20 days of paid leave in their first year. You can apply for leave through the HR portal.

- Benefits of RAG + Prompt Engineering**
- ✓ More accurate and reliable answers
 - ✓ Uses your private / domain data
 - ✓ Reduces hallucinations
 - ✓ Customizable for your use case
 - ✓ Works with latest and internal information
 - ✓ Production ready



- ✓ Track changes
- ✓ Compare results
- ✓ Learn what works
- ✓ Build a reliable prompt

Prompt Versioning

Iterate, improve and keep versions – get better results over time!

Versioning turns prompting from trial and error into a systematic process!

Prompt v1

Jan 1, 2024

Summarize this document.



Result:
Too generic
Missing key points
Not in desired format



Does not meet requirements

Add
more context

Prompt v2

Jan 10, 2024

You are a helpful assistant.
Summarize this document in 5 key points.



Result:
Better!
Captures key points
But still missing tone
and audience context



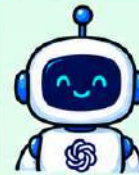
Partially meets requirements

Refine
instructions
+ add format

Prompt v3

Jan 20, 2024

You are a helpful assistant.
Summarize this document in 5 key points
for a non-technical audience. Use simple
language and present the output in a
bulleted list.



Result:
Great!
Accurate, concise
Right tone and format
Ready for production



Meets requirements

Keep iterating and improving!

Better prompts
lead to better
products!



Version. Evaluate. Improve. Deploy.

Prompt versioning = Consistent and reliable results!

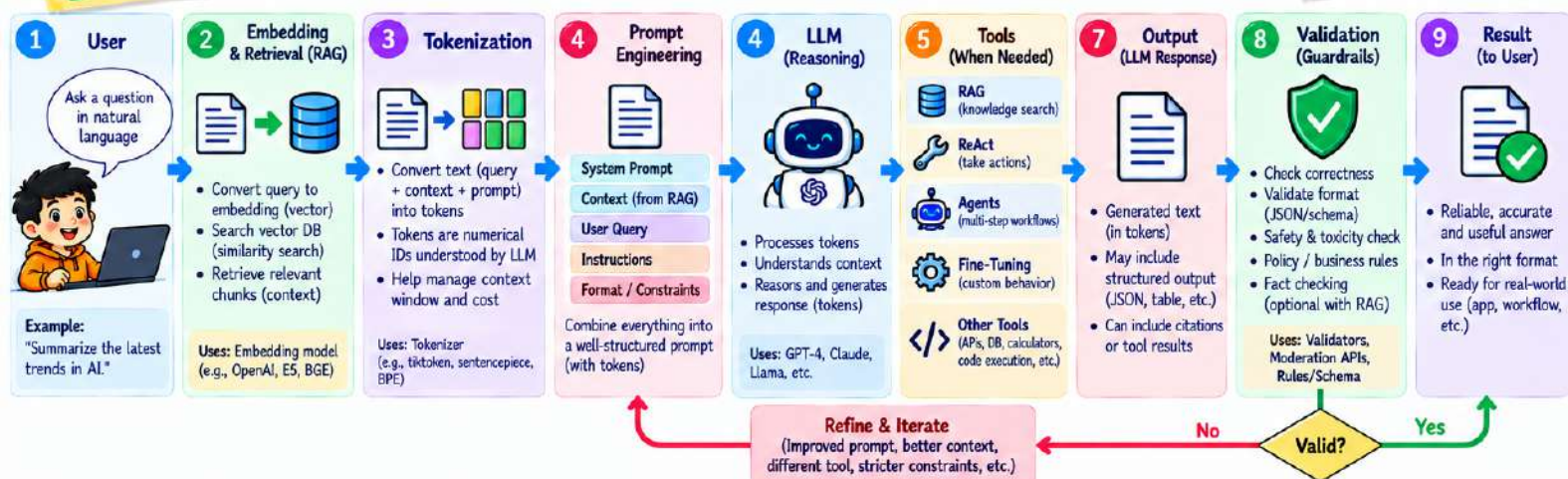
- ✓ Maintain history
- ✓ A/B test versions
- ✓ Use evaluation metrics
- ✓ Find the best version
- ✓ Production ready!

End-to-End AI Application Flow

From User Query to Reliable Result (with Tokenization and Tools)

- ✓ Right data
- ✓ Right tokens
- ✓ Right prompt
- ✓ Right tools
- ✓ Right validation
- ✓ Reliable results!

Think → Retrieve →
Augment → Generate →
Validate → Deliver →
Improve

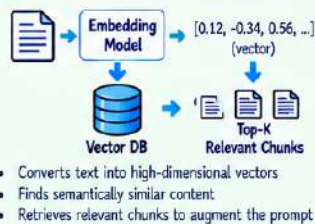


Tokenization in Detail



- Splits text into smaller pieces (tokens)
- Uses methods like BPE (Byte Pair Encoding)
- Same tokenizer is used for input and output
- Tokens are limited by context window (e.g., 128K)
- Affects cost, performance and quality

How Embeddings Work (RAG)



Prompt Template Example

```
<system>
You are a helpful assistant.
</system>
<context>
{retrieved_context}
</context>
<user>
{user_query}
</user>
Respond in JSON format.
```

Tools Examples

- RAG – search internal docs, web
- ReAct – use tools (APIs, code, etc.)
- Agents – plan and execute multi-steps
- Fine-Tuning – domain specific behavior
- APIs – real-time data (weather, stocks)
- DB – query your database
- Code Execution – run calculations and more

What Makes it Reliable?

- ✓ Good retrieval (relevant context)
- ✓ Clear and structured prompts
- ✓ Proper token handling
- ✓ Right tools and actions
- ✓ Validation and guardrails
- ✓ Continuous evaluation
- ✓ Monitoring in production
- ✓ Iterate and improve



Prompt Engineering + RAG + Tokenization + Tools + Validation = Real-World AI Applications

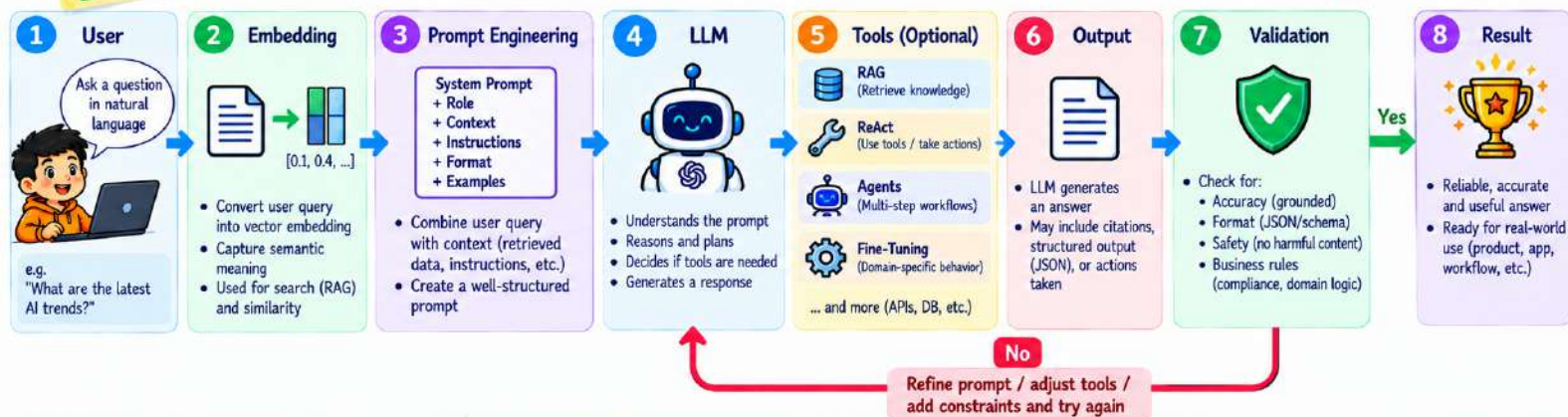
It's a loop, not a one-shot!

AI Application Flow

From User Query to Reliable Result

- ✓ Right data
- ✓ Right prompt
- ✓ Right tools
- ✓ Right validation
- ✓ Reliable results!

Iterate until it's right, then deliver with confidence!



Key Takeaways

- ✓ Embeddings help find the right context.
- ✓ Prompt engineering guides the LLM.
- ✓ Tools (RAG, ReAct, Agents, Fine-Tuning) extend capabilities.
- ✓ Validation ensures reliability.
- ✓ Iterate until it's right, then deliver.



Examples of Tools

- RAG – search internal docs, DB, web
- ReAct – call APIs, run code, take actions
- Agents – plan and execute multi-steps
- Fine-Tuning – customize model behavior



Benefits

- ✓ More accurate answers
- ✓ Uses your own data (private knowledge)
- ✓ Handles complex, real-world tasks
- ✓ Safe, reliable and production-ready
- ✓ Scales with your business needs



Big Picture

Prompt Engineering + RAG + Tools + Validation = **AI Engineering**

Better Prompts. Smarter Systems. Real Impact.

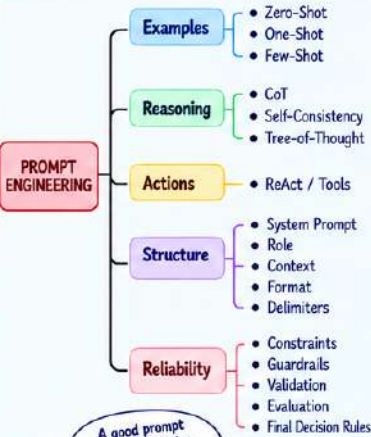
Same LLM.
Better Prompts.
Bigger Possibilities!

Prompt Engineering & AI Engineering

From Better Prompts to Real-World Applications

Ideas → Prompts
Prompts → Systems
Systems → Real Impact!

1 Prompt Engineering Framework



A good prompt is the foundation of a great AI system!

2 When to Use What?

	Situation / Need	Use
	Simple task	Zero-Shot
	Specific pattern / format	Few-Shot
	Complex reasoning	CoT
	Need more robust reasoning	Self-Consistency
	Need live data / API / action	ReAct + Tools
	Need exploration / backtracking	Tree-of-Thought
	Need fresh / private knowledge	RAG
	Need stable behavior at scale	Fine-Tuning

3 Prompt Engineering Examples

Zero-Shot	"Summarize this article."
One-Shot	Q: What is 2+2? A: 4 Q: What is 5+3? A:
Few-Shot	Classify the sentiment: Text: "I love this!" → Positive Text: "This is bad." → Negative Text: "Amazing experience!" →
CoT (Chain-of-Thought)	Solve step by step: If a car travels 60 km in 1 hour, how far in 3 hours? Let's think step by step...
ReAct (with Tools)	What's the current weather in Delhi? (Use a search tool)
Structured Output	Return the result in JSON format: { "name": "", "age": 0 }

4 Prompt Structure (Template)

System Prompt	→ You are a helpful assistant.
Role	→ Act as a travel planner.
Context	→ The user is visiting India for 7 days.
Task	→ Create a day-wise itinerary.
Format	→ Return in a table with columns: Day, Place, Activities, Tips.
Delimiters	→ Use ### to separate context.

5 Reliability & Safety

Constraints	→ Set boundaries (e.g., don't answer medical/legal advice).
Guardrails	→ Input/output filtering, toxicity checks.
Validation	→ Schema / format / fact checking.
Evaluation	→ Use test cases and metrics.
Final Decision Rules	→ Deploy only if it meets quality, safety and business requirements.

6 End-to-End AI Engineering Flow

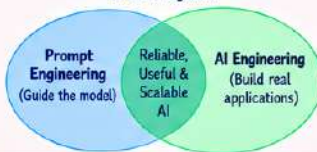


7 AI Engineering Connection

Prompt Engineering is one part of a bigger picture — AI Engineering.

- ✓ Prompt Engineering → Controls how the model behaves
- ✓ RAG → Gives fresh and private knowledge
- ✓ Tools → Lets the model take real actions
- ✓ Agents → Helps in complex, multi-step workflows
- ✓ Structured Output → Makes it reliable for applications
- ✓ Validation → Ensures safety, accuracy and compliance
- ✓ Production → Delivers real value to users at scale

All Work Together!



Great prompts make great answers.
Great AI engineering makes real impact!

8 Key Takeaways

- ✓ Prompt engineering and RAG are complementary, not alternatives.
- ✓ Use the right technique for the right problem.
- ✓ Add validation and guardrails for production.
- ✓ Combine prompts, knowledge, tools and agents to build powerful applications.
- ✓ Test, iterate and monitor continuously.
- ✓ The goal is not just a good answer, but a safe, reliable and useful product.



Better Prompts. Smarter Systems. Real-World Impact.

- ✓ Use tools when needed
- ✓ Get real-time information
- ✓ Handle complex tasks
- ✓ More accurate answers

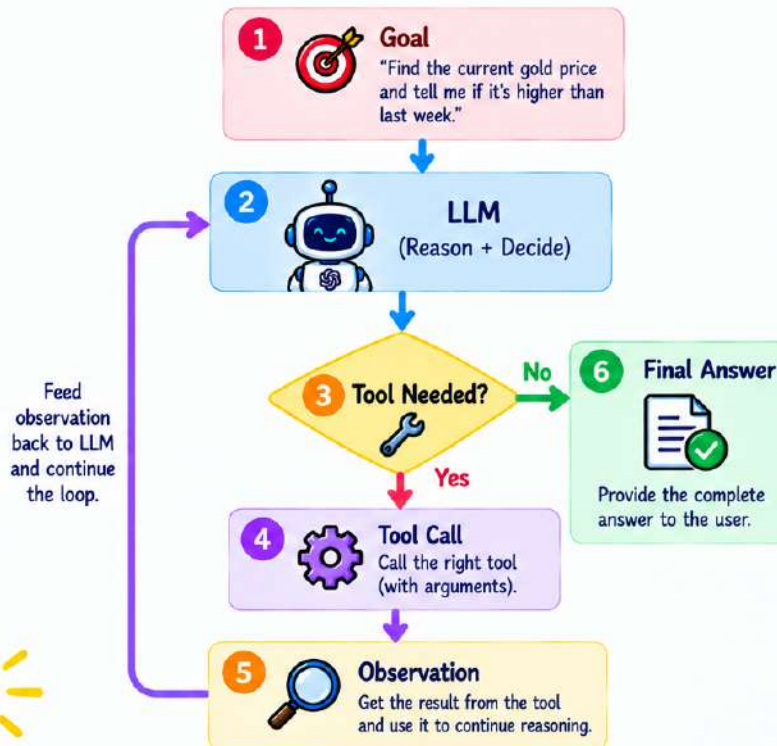
What are Tools?

-  **Web Search**
(latest information)
-  **Database Query**
(internal data)
-  **APIs**
(weather, maps, etc.)
-  **Calculations**
(math, conversions)
-  **File Access**
(read/write files)
- ... and more!

LLM + Tools
= Superpowers
in real-world tasks!

ReAct + Tools

Think → Act → Observe → Repeat → Solve!



ReAct makes LLMs more capable by letting them use tools to get real information and take actions!

LLM can decide
when to use tools
and what to do next!

Example: Live Data with ReAct

User: What is the current gold price and is it higher than last week?

Thought: I need to get the latest gold price. I should use a web search tool.

Action: web_search("current gold price in USD per ounce")

Observation: Gold price is \$2,505/oz (as of Oct 6, 2024).

Thought: Now I need last week's price to compare. I'll search for last week's gold price.

Action: web_search("gold price Sept 29 2024 USD per ounce")

Observation: Gold price was \$2,470/oz (on Sept 29, 2024).

Thought: Current price (2,505) is higher than last week (2,470). I can now answer.

Final: The current gold price is \$2,505/oz, which is higher than last week's price of \$2,470/oz.

- ✓ More accurate answers
- ✓ Real-time information
- ✓ Can handle complex tasks
- ✓ Production-ready pattern!

- ✓ Accurate answers
- ✓ Less hallucinations
- ✓ Safe and reliable
- ✓ Ready for production!

RAG + Prompt + Validation

Better context + better prompts + safety checks = reliable answers!

RAG + Prompt + Validation
work together to turn
knowledge into reliable,
useful answers!

1 User Query



User asks a question
in natural language.

2 Retriever



Searches your knowledge
base (documents, DB,
APIs, etc.) for relevant
information.

e.g. Vector DB
(Pinecone, Qdrant, FAISS)

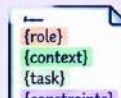
3 Relevant Context



Top-k relevant chunks
are retrieved.

e.g. Company HR policy
document, latest guidelines

4 Prompt Template



Combine user query +
retrieved context with a
well-structured prompt.

e.g. System prompt,
instructions, format rules

5 LLM



Generates an answer
using the prompt
and retrieved context.

e.g. GPT-4, Claude,
Llama, etc.

6 Schema / Policy Validation



Validate the output
for schema, content rules,
safety, and policy
compliance.

e.g. JSON schema,
PII check, toxicity filter,
business rules

7 Answer



Return a safe, accurate
and useful answer
to the user.

"As per our company
policy, you are eligible
for 20 days of paid leave
in a year...."



RAG brings the knowledge. Prompt gives the direction. Validation builds trust.

- ✓ Right information
- ✓ Right format
- ✓ Right guardrails
- ✓ Reliable output